

Stacks and Queues

Queue

- enqueue: places an element at the tail of the queue
- dequeue: removes the next available element from the head of the queue
- isEmpty: returns true or false depending on whether the underlying data structure has any remaining elements

Stack

- push: places an element at the top of a stack
- pop: removes an element from the top of a stack
- isEmpty: returns true or false depending on whether the underlying data structure has any remaining elements

Linked Lists

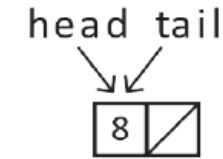
- Linked lists are well suited for both queues and stacks
- adding and removing elements is simply a matter of creating a node and setting a reference or removing a reference.
- It is possible to track both ends of the data structure

```

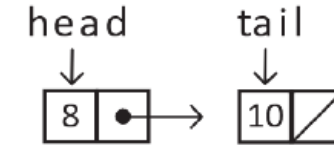
1  class QueueList
2      Node head
3      Node tail
4
5      function enqueue(value)
6          set newNode to new instance of a node
7          set newNode.Value to value
8          if head is null
9              set head to newNode
10             set tail to newNode
11         else
12             set Tail.Next to newNode
13
14     function dequeue()
15         set value to head.Value
16         set head to head.Next
17         return value
18
19     function isEmpty()
20         return head equals null

```

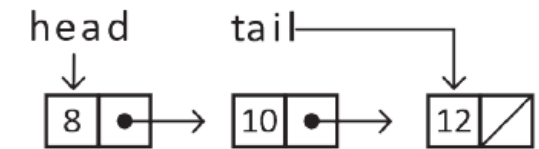
① Enqueue value 8



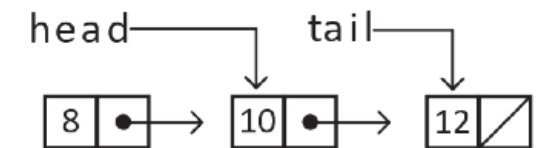
② Enqueue value 10



③ Enqueue value 12



④ Dequeue - returns 8



```

1  class StackList
2      Node top
3
4      function push(value)
5          set newNode to new instance of a node
6          set newNode.Value to value
7          if top is null
8              set top to newNode
9          else
10             set newNode.Next to top
11             set top to newNode
12
13     function pop()
14         set value to top.Value
15         set top to top.Next
16         return value
17
18     function isEmpty()
19         return top equals null

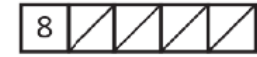
```

- Most operations on linked lists are $O(n)$ due to the fact we have to traverse lists one node reference at a time. In this special case of queues and stacks, no traversal is necessary as long as we maintain a reference to the head or tail.
- No matter how many elements we enqueue or push, we only ever care about one or two nodes at any given time. The result is an elegant solution that runs in $O(1)$ time for most operations.

Arrays

```
1  class QueueArray
2      Array data
3      Integer headPos
4      Integer tailPos
5      set headPos to 0
6      set tailPos to -1
7
8      function enqueue(value)
9          if tailPos + 1 equals length(data)
10             Copy values into larger array D
11             set data to D
12             set tailPos to tailPos + 1
13             set data[tailPos] to value
14
15     function dequeue()
16         set value to data[headPos]
17         set headPos to headPos+1
18         return value
19
20     function isEmpty()
21         return tailPos < headPos
```

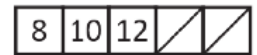
- ① Enqueue value 8
headPos= 0 tailPos= 0



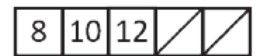
- ② Enqueue value 10
headPos= 0 tailPos= 1



- ③ Enqueue value 12
headPos= 0 tailPos= 2



- ④ Dequeue - returns 8
headPos= 1 tailPos= 2



```
1  class StackArray
2      Array data
3      Integer topPos
4      set topPos to -1
5
6      function push(value)
7          if topPos + 1 equals length(data)
8              Copy values into larger array D
9              set data to D
10             set topPos to topPos + 1
11             set data[topPos] to value
12
13     function pop()
14         set value to data[topPos]
15         set topPos to topPos - 1
16         return value
17
18     function isEmpty()
19         return topPos < 0
```